Can LLMs Do Causal Inference?

Enqi Dai Ziyu Huang Junchi Wang Candy Wu Jiaqi Zhu
Mentor: Professor Jelena Bradic

CornellBowers
College of Computing + Information Science

Question and main message
Can a general-purpose LLM estimate the **average treatment effect** from observational data without fine-tuning? We treat the model as a protocol-conditioned causal calculator, not an oracle.

Design
The study combines **8 simulation engines**, **4 method-inducing protocols**, strict JSON output parsing, repeated Monte Carlo evaluation, and side-by-side comparison with classical causal estimators.

What is evaluated
Point estimation, uncertainty calibration, and diagnostic behavior: bias, RMSE, empirical coverage, CI width, overlap warnings, weak-support stability, protocol sensitivity, and PSID transfer.

Research Question, Estimand, and Assumptions

Estimand. The target is the population average treatment effect

$$\tau = \mathbb{E}\{Y(1) - Y(0)\}.$$

The LLM never observes the potential outcomes directly. It receives observational summaries and must output an estimate of τ, a 95% CI, and diagnostics. Identification depends on:

- **Consistency / SUTVA**: observed outcomes match the realized treatment condition.
- **Unconfoundedness**: $\{Y(1), Y(0)\} \perp A \mid X$.
- **Overlap / positivity**: $0 < e(X) = \Pr(A = 1 \mid X) < 1$.

This project evaluates whether an LLM can *execute a specified estimator, respect its assumptions, and communicate uncertainty*, rather than whether it discovers causality on its own.

Simulation Engines and Stress Tests

E	Primary stressor and statistical purpose
1	Linear confounding baseline with constant effect.
2	Nonlinear outcome model; tests regression misspecification.
3	Interaction-heavy outcomes and heterogeneous treatment effects.
4	Sparse high-dimensional confounding with many irrelevant covariates.
5	Near-positivity violations and unstable weighting regimes.
6	Nonlinear treatment assignment and nonlinear confounding.
7	Missing outcomes under MCAR/MAR mechanisms.
8	Measurement error and heavy-tailed outcome noise.

Ground truth. Each engine creates repeated finite samples with a known true ATE, so estimation error is observable. Results can be stratified by causal difficulty rather than reduced to a single average. This matters because an estimator that succeeds in a linear setting may fail under weak overlap, heterogeneous effects, high dimensionality, missingness, or heavy-tailed noise. The design therefore distinguishes arithmetic errors from genuine causal difficulty.

Method-Inducing ICL Protocols

ID	Target	Prompt construction
A	Diff-in-means	Treated/control n, outcome means, and SDs. The LLM computes the arm contrast, SE, and 95% CI.
B1	RA from micro-data	A compact subset of rows (A, Y, X) so the model reasons through covariate adjustment using raw examples.
B2	RA from compressed output	OLS summary output: sample size, covariate count, R^2, coefficient on A, and SE. The LLM converts the treatment coefficient into an adjusted ATE.
C	Propensity stratification	PS bins with weights, treated/control counts, within-bin means, and SDs. The LLM computes weighted bin contrasts and flags weak cells.
D	AIPW / doubly robust	Nuisance quantities $(A, Y, \hat{e}, \hat{\mu}_0, \hat{\mu}_1)$ or aggregates. The LLM applies the AIPW score and reports overlap concerns.

Shared ICL structure. Every prompt has: (i) a fixed task block defining the ATE; (ii) a protocol-specific reminder; (iii) serialized sufficient summaries; (iv) overlap-warning rules; and (v) a JSON-only response contract. The prompt is *designed to induce an estimator*: the LLM receives the sufficient ingredients for a target procedure rather than being asked to improvise from raw data. Few-shot examples reinforce arithmetic, valid JSON formatting, cautious behavior under weak overlap, and returning `null` or warnings when empty cells or missing nuisance predictions prevent valid estimation. This makes protocol comparisons interpretable: Protocol A tests simple arithmetic, B tests covariate adjustment logic, C tests overlap-aware aggregation, and D tests doubly robust computation.

Classical Causal Estimators and Metrics

Benchmarks. Difference-in-means ignores confounding; regression adjustment relies on outcome-model adequacy; propensity methods rely on overlap and weight stability; and AIPW remains sensitive to nuisance estimation and support despite double robustness.

$$\hat{\tau}_{DM} = \bar{Y}_1 - \bar{Y}_0,$$
$$\hat{\tau}_{Strat} = \sum_k \hat{w}_k(\bar{Y}_{1k} - \bar{Y}_{0k}),$$
$$\widehat{\tau}_{AIPW} = n^{-1} \sum_i \left[\hat{\mu}_1 - \hat{\mu}_0 + \frac{A_i(Y_i - \hat{\mu}_1)}{\hat{e}_i} - \frac{(1 - A_i)(Y_i - \hat{\mu}_0)}{1 - \hat{e}_i} \right].$$

Scoring. Point estimates are scored by bias, absolute bias, RMSE, and tail behavior. Intervals are scored by empirical coverage, average width, implied SE, and calibration against empirical SD. Diagnostics include PS quantiles, empty bins, max weights, and effective sample size.

Traps, Diagnostics, and Guardrails

- **Trap variables**: post-treatment variables test bad adjustment; treatment-only predictors test confounder recognition.
- **Overlap diagnostics**: empty PS cells, high weights, or low effective sample size trigger caution.
- **Output guardrails**: JSON parsing, CI sanity checks, and arithmetic recomputation make outputs auditable.
- **Interpretive guardrail**: disagreement across protocols is an estimator-sensitivity signal.

End-to-End Procedure

1. Generate observational data (X, A, Y) from a known engine; hide true potential outcomes.
2. Clean and serialize: reorder columns, round values, optionally standardize or subsample.
3. Build protocol summaries: arm moments, raw rows, compressed regression output, PS strata, or AIPW nuisance tables.
4. Insert summaries into a prompt with estimator instructions, assumptions, and diagnostic thresholds.
5. Query the LLM with a strict JSON schema requiring $\hat{\tau}$, 95% CI, method label, assumptions, and warnings.
6. Parse and validate: check numerical fields, CI containment, overlap flags, and arithmetic.
7. Compare with truth and classical benchmarks; aggregate by engine, ATE, and overlap regime.

This pipeline evaluates both **statistical behavior** and **prompt-interface design** within one reproducible framework. Simulated tasks use repeated known-truth datasets so that point error and interval coverage are measurable. The PSID task has unknown truth, so it is assessed through consistency with classical benchmarks, overlap quality, protocol stability, and whether the LLM avoids unsupported causal claims.

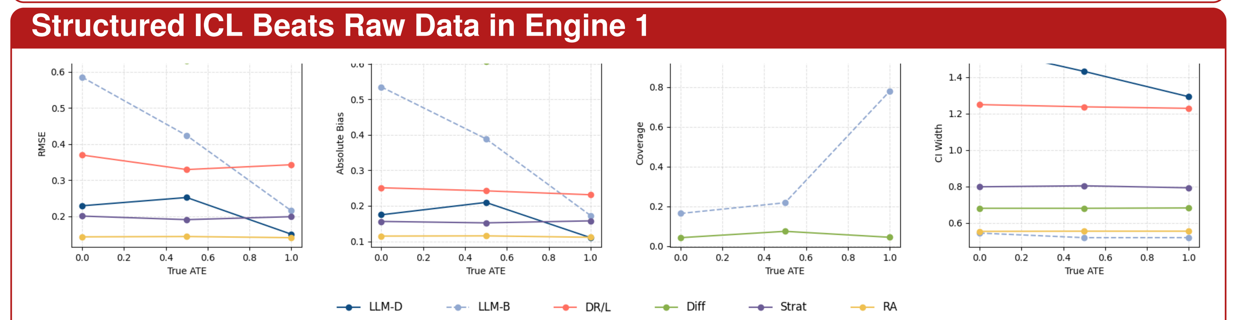
Prompt Schema and Output Contract

Prompt contents. Each evaluation prompt contains a task statement, protocol-specific data block, estimator reminder, optional few-shot exemplars, and a strict output contract.

- Required fields: `ate_hat`, `ci95`, `method`, `assumptions`, `overlap_warning`, and `diagnostics`.
- Hard checks: CI contains the point estimate; numbers parse; warnings match diagnostics.
- Edge cases: missing nuisances, empty strata, or impossible intervals should produce caution rather than a confident estimate.

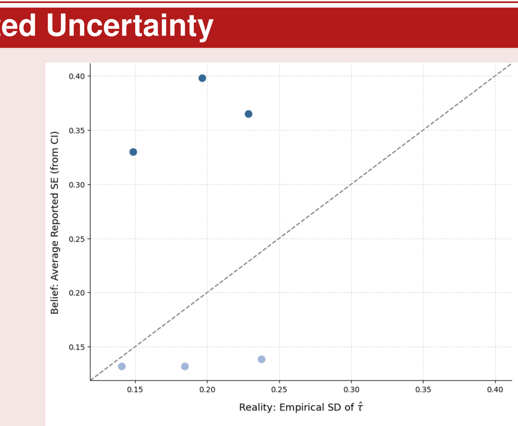
The prompt is an estimator interface: it constrains the model to compute, report, and diagnose a pre-specified causal procedure. Format failures are excluded from numerical scoring, while valid outputs are checked for CI ordering, CI containment of the point estimate, plausible method labels, and diagnostic consistency. This avoids giving credit for persuasive but non-auditable prose.

Structured ICL Beats Raw Data in Engine 1



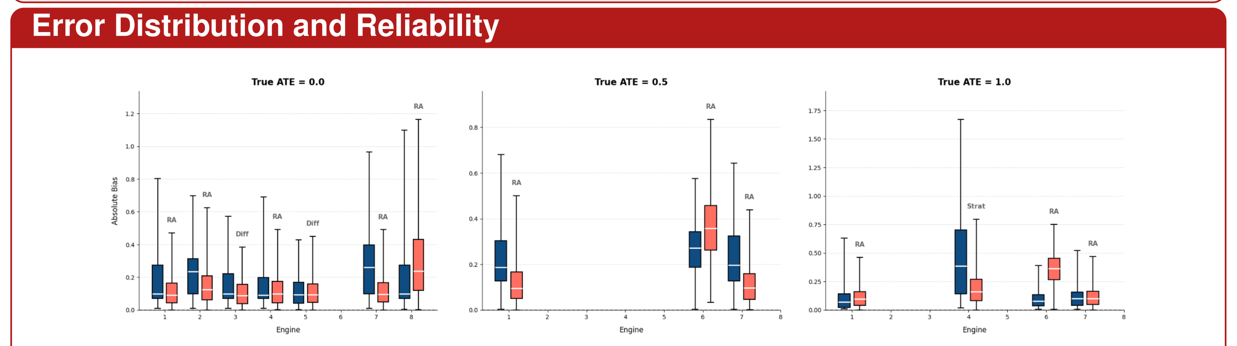
Interpretation. In the simplest linear-confounding engine, Protocol D reduces RMSE and absolute bias relative to raw-data prompting while maintaining high coverage. The practical lesson is methodological: better LLM causal inference does not come from pasting more rows into the prompt, but from supplying the *right statistical summaries*. This result motivates using ICL to induce specific estimators rather than relying on unstructured reasoning.

Calibration of Reported Uncertainty



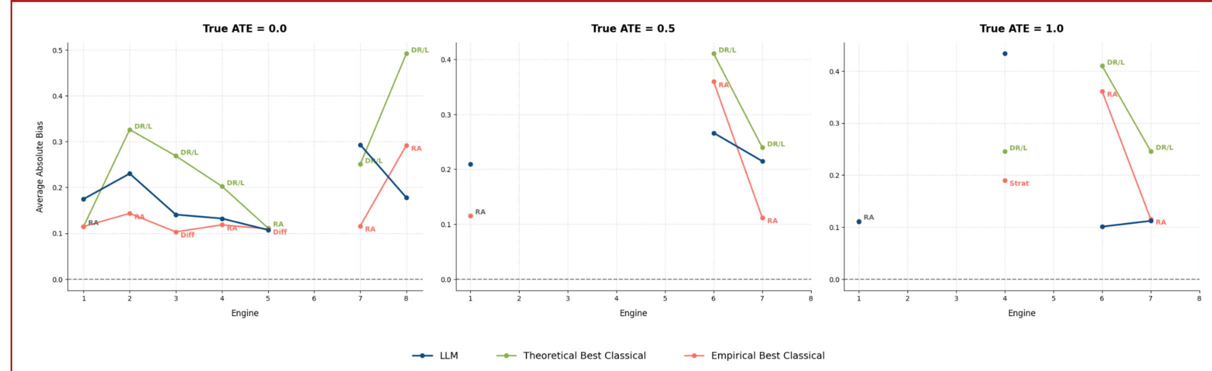
Interpretation. Points below the 45-degree line under-report uncertainty; points above it over-report. The raw-data baseline is often overconfident. LLM-D is conservative: intervals are not always efficient, but they more closely respect empirical variation. Mild conservatism is preferable to overconfident causal claims when treatment assignment is observational.

Error Distribution and Reliability



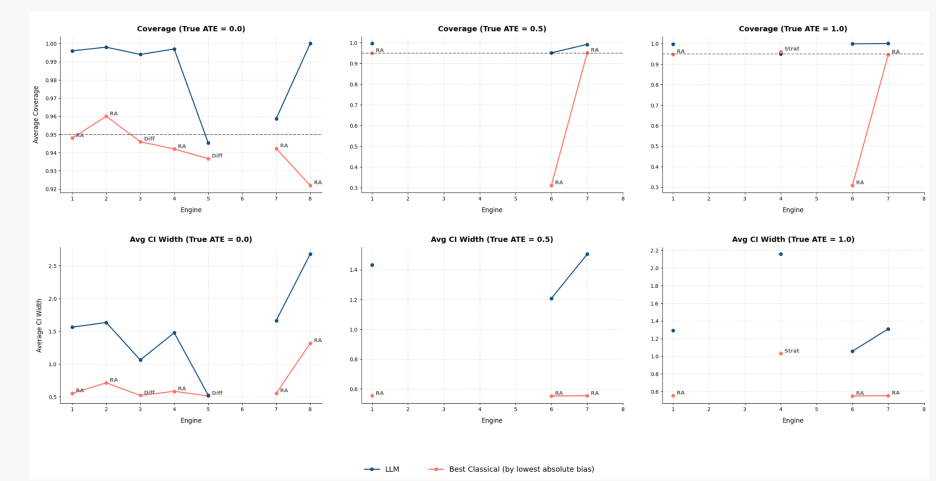
Interpretation. Median behavior can be competitive, yet longer LLM whiskers reveal occasional large mistakes. A favorable average score is not enough for deployment; one also needs tail-risk monitoring, schema-failure tracking, implausible-interval detection, and independent recomputation.

Across-Engine Bias Patterns



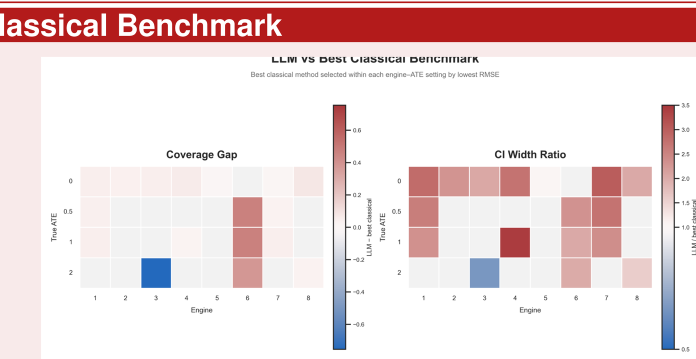
Interpretation. LLM-D is most competitive when the true effect is zero or modest. As the true ATE grows, estimates often shrink toward zero, producing systematic under-estimation in harder regimes. Engine 6 is a notable exception: nonlinear confounding hurts standard linear benchmarks, while the structured LLM protocol remains competitive.

Coverage and CI Width



Interpretation. Protocol D often attains strong empirical coverage by widening confidence intervals. This is conservative: it reduces efficiency in easy settings but protects against under-coverage when structural assumptions are mismatched or overlap is weak. Thus uncertainty is a first-class evaluation target, not an afterthought.

LLM vs Best Classical Benchmark



How to read this panel. The left heatmap compares coverage gaps; the right compares CI-width ratios. Red cells usually indicate more conservative LLM intervals; blue cells indicate possible under-coverage or overly narrow intervals. The central trade-off is explicit: the LLM often buys reliability with wider intervals.

Main Simulation Conclusions

1. **Prompt structure dominates raw prompting.** Method-inducing summaries outperform raw-row prompts.
2. **The LLM behaves like an estimator interface.** Performance depends on which estimator logic the prompt induces.
3. **Conservative uncertainty is common.** Wider intervals improve coverage but can hide low precision.
4. **Large-effect shrinkage is the main point-estimate failure mode.**
5. **Safe use requires redundancy.** Classical benchmarks, overlap diagnostics, and arithmetic verification remain essential.

Methodological interpretation. The LLM can implement a causal recipe when the prompt packages the estimator, diagnostics, and output constraints; it should not be granted authority over identification assumptions.

Limitations and Next Steps

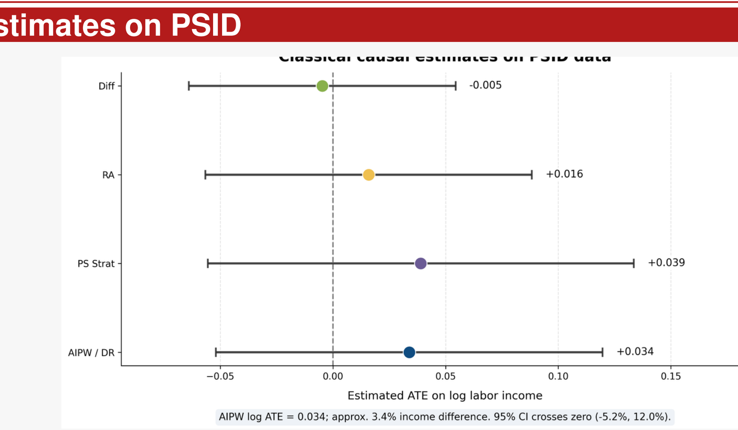
- Add stronger arithmetic validators and automatic recomputation checks.
- Compare more flexible nuisance learners for AIPW and PS estimation.
- Stress-test prompt sensitivity, non-random missingness, and subgroup effects.
- Separate estimator-execution errors from identification failures in real data.

PSID Observational Study

Application question. What is the effect of human-capital-enhancing participation (job training, vocational training, or higher education) on later log labor income?

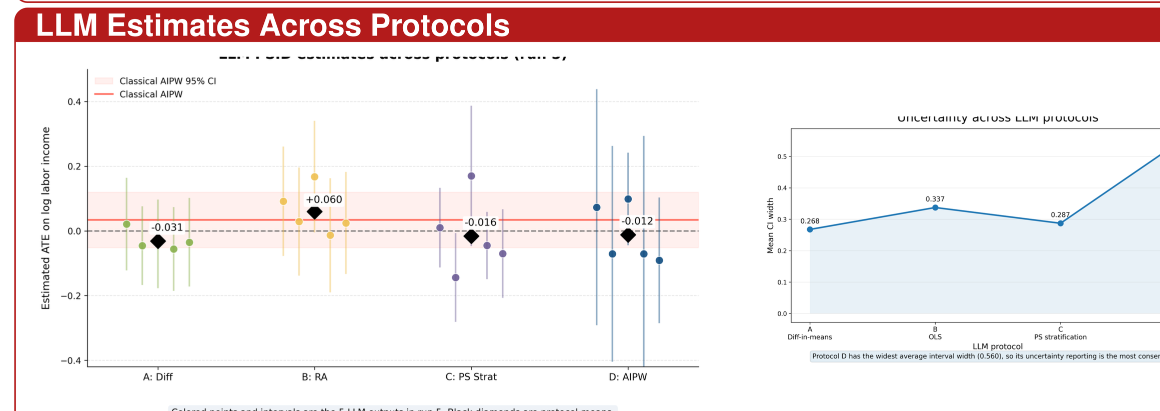
- Baseline covariates are measured in 2011, treatment in 2013, and outcome in 2023.
- The cleaned sample contains 6,096 observations: 4,240 treated and 1,856 controls.
- The outcome is log labor income; the treatment combines job training, vocational training, and higher education participation.
- Covariates include demographics, education, labor-market history, family structure, baseline economic variables, and food assistance.
- The real-data task has no known truth, so evaluation relies on overlap, classical benchmarks, protocol consistency, and whether the model avoids overclaiming.

Classical Estimates on PSID



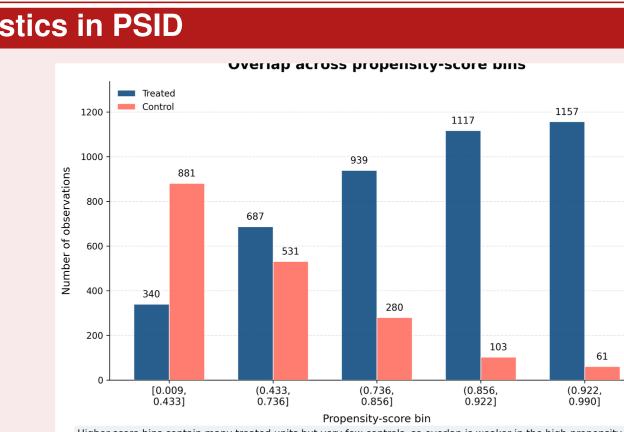
Interpretation. All four classical intervals cross zero. The AIPW estimate is about 0.034 log points, loosely a 3.4% income difference, but its interval includes negative and positive effects. The classical analysis therefore supports a cautious conclusion: the average effect is small and statistically uncertain.

LLM Estimates Across Protocols



Interpretation. Protocol means remain close to zero, broadly consistent with classical analysis, but they are not identical. Protocol D reports the widest average intervals, matching the simulation pattern of cautious uncertainty. This reinforces the central message: the LLM output is a **protocol-dependent estimator**, not a single oracle answer.

Overlap Diagnostics in PSID



Interpretation. High-propensity bins contain many treated observations but very few controls. The upper bins have the weakest comparison support, so IPW and AIPW can become unstable. A trustworthy LLM protocol should therefore widen its interval, issue an overlap warning, or recommend sensitivity analysis rather than pretending the design is well supported.

Practical Conclusions

- Use LLMs to **execute and explain** a pre-specified causal protocol, not to invent one from raw data.
- Prefer structured summaries that expose estimator ingredients, assumptions, and failure modes.
- Require machine-checkable output, overlap diagnostics, and independent arithmetic verification.
- Report disagreement across protocols; it is an estimator-sensitivity signal.
- Treat wide intervals as caution, not proof; pair interval width with bias, overlap, and support diagnostics.
- Final causal claims still rest on unconfoundedness, positivity, measurement quality, and domain context.
- Next steps: stronger validators, flexible nuisance learners, prompt-sensitivity checks, and subgroup-specific effects.

Take-home

LLMs are useful here only when their statistical role is constrained.

1. Induce estimators
Protocols A-D encode estimator logic; the prompt design is part of the statistical method, not just formatting.

2. Score uncertainty
Judge bias/RMSE together with coverage, CI width, implied SE, and calibration against Monte Carlo variability.

3. Diagnose support
Weak overlap, empty strata, large weights, missing nuisances, and protocol disagreement are substantive results.

4. Verify every answer
Use LLMs to execute and explain a pre-specified recipe; pair outputs with benchmarks and recomputation.